

Library Management System

Objective:

Design and implement a Library Management System using Java 8. The system will allow users to manage books and members, issue and return books, and generate reports using the Java Stream API. The application will use JavaFX for the UI, JDBC for database interactions, and JUnit for unit testing.

This project is intended to be developed by a team of **2 to 3 members**.

Requirements:

Functional Requirements:

Book Management (CRUD):

- Add a book
- Update book details(excluding availability)
- Update book availability
- View all books

Member Management (CRUD):

- Register a new member
- Update member details
- View all members

Issue and Return Books:

- Issue a book to a member
- Return a book
- Prevent issuing already issued books

Reports (using Java 8 Streams):

- List of overdue books
- Count of books per category
- List of members with active issued books

Exception Handling:

- Handle invalid inputs (e.g., null, empty)
- Handle database errors (e.g., connection failure)

- Handle logical errors (e.g., issuing already issued books)

Transaction Handling:

- Issue and return operations must be transactional (either fully complete or rollback)

JavaFX UI:

- Simple form to add a book
- Simple form to add a member
- View all books
- View all members
- Issue of Book to a Member
- Return of Book
- Show all issued records
- Selection of Member shows issued Books to the member(optional)
- Selection of Books shows used Members(optional)

JUnit Tests:

- Unit tests for every service logic (e.g., issue/return validation)
- Unit tests for DAO operations

Database Schema:

Table: books and books_log(No Auto Increment, Constraints)

- BookId INT PRIMARY KEY AUTO_INCREMENT
- Title VARCHAR(255) NOT NULL
- Author VARCHAR(255) NOT NULL
- Category VARCHAR(100) NOT NULL
- Status CHAR(1) NOT NULL [Active(A) / Inactive(I)] (book-condition)
- Availability CHAR(1) NOT NULL ['Available(A) / Issued(I)'] (book availability for issue)

Table: members and members_log(No Auto Increment, Constraints)

- MemberId INT PRIMARY KEY AUTO_INCREMENT
- Name VARCHAR(255) NOT NULL
- Email VARCHAR(255) NOT NULL UNIQUE
- Mobile INT NOT NULL UNIQUE
- Gender CHAR(1) NOT NULL [Male(M)/Female(F)]

- Address VARCHAR(255) NOT NULL

Table: issue_records and issue_records_log(No Auto Increment, Constraints)

- IssueId INT PRIMARY KEY AUTO_INCREMENT
- BookId INT NOT NULL
- MemberId INT NOT NULL
- Status CHAR(1) NOT NULL ['Issued(I) / Returned(R)']
- IssueDate DATE NOT NULL
- ReturnDate DATE

Technology Stack:

- Java 8
- JavaFX (UI)
- JDBC (Database Connectivity)
- JUnit 4
- MySQL

Key Implementation Details:

OOP Principles:

- POJO classes: Book, Member, IssueRecord
- Encapsulation, abstraction, and separation of concerns via DAO and service layers
- Custom exceptions extending `Exception`

Note on Task Assignment:

To ensure modular development and avoid conflicts:

- A developer assigned to any UI functionality (e.g., Add Book UI) **must not work** on any other UI functionality (e.g., Add Member UI).
- A developer working on a UI functionality (e.g., Add Book UI) **must not work** on its corresponding backend API (e.g., Add Book API)
- This policy applies similarly to Create, Update and View functionalities to ensure proper division of responsibilities and clean integration boundaries.